Object Oriented Programming

10636212

Dr. Ashraf Armoush

© 2020 Dr. Ashraf Armoush

Chapter 01: Introduction to OOP

© 2020 Dr. Ashraf Armoush

History of Java

- **Java** is a programming language originally developed at **SUN Microsystems**.
  - Originally for intelligent consumer-electronic devices (*e.g. microwave ovens and remote controls*) [The first public release Java1.0 in 1995]
  - Then used for creating Web pages with *dynamic content (interactivity* and *animations*) using Applets.
  - Now also used for:
    - Develop large-scale enterprise applications
    - Enhance WWW server functionality
    - Provide applications for consumer devices (e.g., *cell phones*, *smartphones*, *television set-top boxes* , and more)
      - **3 Billion** Mobile Phones Run Java [2013]
      - **9 Million** Java Developers Worldwide [2013]
  - One of the most popular programming languages in use
- **Java SE** vs. **Java EE**
  - **EE**: **enterprise edition** (web services, distribution, RMI, …)
  - **SE**: **standard edition** (stand alone app.)

The Java Programming Language Platforms

- **Java Platform, Standard Edition (Java SE)**
  - Java SE's API provides the **core functionality** of the Java language.
  - In addition to the core API, the Java SE platform consists of a virtual machine, development tools, deployment technologies, and other class libraries and toolkits commonly used in Java technology applications.
- **Java Platform, Enterprise Edition (Java EE)**
  - Is built on top of the Java SE platform.
  - Provides an API and runtime environment for developing and running large-scale, multi-tiered, scalable, reliable, and secure **network applications**.

    موثوقة قابلة للتطوير متعددة المستويات واسعة النطاق
- **Java Platform, Micro Edition (Java ME)**
  - Provides an API and a small-footprint virtual machine for running Java applications on embedded and mobile devices.
- **JavaFX ()**
  - JavaFX is a platform for creating **R**ich **I**nternet **A**pplications (RIA) using a lightweight user-interface API
  - *Merged to Java SE8*

Java Class Libraries

- Classes
  - Include *methods* that perform tasks
    - Return information after task completion
  - Used to build Java programs
- Java contains powerful class libraries
  - Known as Java **API**s (**A**pplication **P**rogramming **I**nterfaces)

Why Use Java?

- Portable
- Easy
- Object Oriented
- Secure
- High Performance
- Robust

Portability

- Java is as **interpreted** language
  - Computer that wants to run Java must have a program to convert java code to native machine code.
  - Java compiler converts the source code to bytecodes
  - The bytecode is executed inside the Java Virtual Machine (JVM).

  ➔ Java programming language bytecodes can be executed on any platform.

- Java language does not allow a particular machine to implement different sizes for fundamental types (*integers, bytes*)

Easy

- The syntax is mostly derived from **C** and **C++**
- Eliminates problematic language items:
  - No more pointers
  - No preprocessor like #include or #define
  - Automatic garbage collection - memory management handled by **JVM**
  - Array index limit checking
  - No global functions or variables

- Provides powerful class libraries

Object Oriented

- Java was designed to be **O**bject **O**riented (OO) from the ground up.
- Object: *is a software component that contains a collection of data fields and related methods to model a real-world item.*
- Objects can be thought of as wrapping their data within a *set of functions designed to ensure that the data is used appropriately*,
- The object oriented approach is:
  - More flexible
  - More understandable – it models the real world
  - More maintainable.
  - Encourage source code reusability.
- No coding outside of class definitions

Object Oriented

- **O**bject-**O**riented **D**esign (OOD)
  - Models real-world objects
  - Models communication among objects
  - *Encapsulates* attributes and operations (behaviors)
    - Information hiding
    - Communication through well-defined interfaces
- Object-oriented language
  - Programming in object oriented languages is called *object-oriented programming* (*OOP*)
  - e.g. Smalltalk, Java, Python

OO Concepts and Terminology

- **Encapsulation** :

  "It is the mechanism that binds together code and the data manipulates, and keeps both safe from outside interference and misuse"

  - In Java, the basis of encapsulation is the class
  - The purpose of a class is to encapsulate complexity
    - (*Hiding the complexity of the implementation inside the class*).
  - Hiding data members and member functions (methods) implementations into packages called Objects (e.g. Queue)

OO Concepts and Terminology (cont.)

- **Abstraction**

  " Viewing bigger objects rather than details"

  - Humans manage complexity through abstraction.
    - People do not think of car as a set of thousands of individual parts. They think of it as a well-defined object with its unique behavior.
  - A powerful way to manage abstraction is through the use of hierarchical classification (*Hierarchical abstraction of complex system can also be applied to computer programming*)
    - The **data** from a traditional process-oriented program can be transformed by abstraction into its component **objects**.
    - A **sequence of process steps** can become a collection of **messages** between these Objects.
    - Each object describes its unique behavior.
    - You can treat these objects as concrete entities that respond to messages telling them do something.

OO Concepts and Terminology (cont.)

- **Inheritance**

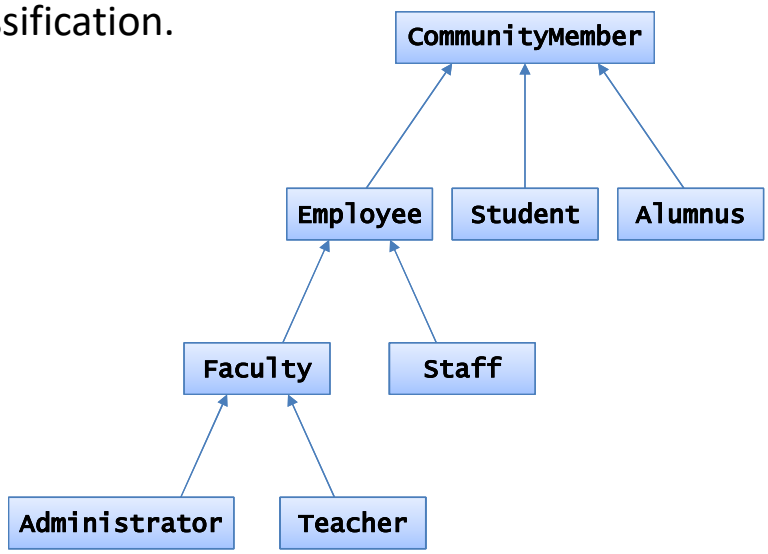
  "The process by which one object acquires the properties of another object. "

  – classes can inherit other classes members without rewriting them (e.g. ExtendedQueue)
    - an object need only define those qualities that make it unique within its class.
    - It can inherit its general attributes from its parent.
    - It is the inheritance mechanism that makes it possible for one object to be a specific instance of a more general case.

  – Inheritance interacts with encapsulation as well.
    - If a given class encapsulates some attributes, then any subclass will have the same attributes plus any that it adds as part of its specialization.

OO Concepts and Terminology (cont.)

– most knowledge is made manageable by hierarchical classification.



– without the use of hierarchies, each object would need to define all of its characteristics explicitly.

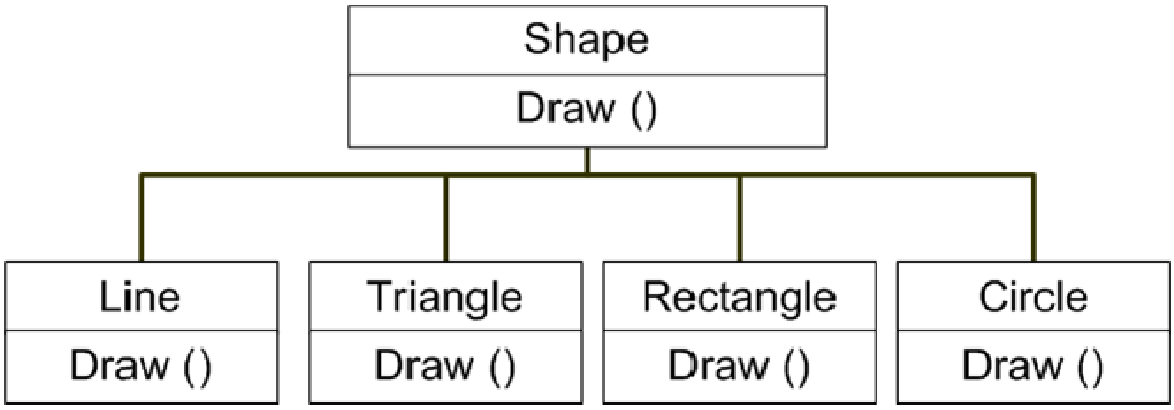
OO Concepts and Terminology (cont.)

- **Dynamic Binding (*Polymorphism*)**

  "feature that allows one interface to be used for a general class of actions. The specific action is determined by the exact nature of the situation"

  - ***Polymorphism*** (from **Greek**, meaning "***many forms***")
  - ***Polymorphism*** and Virtual functions allow late binding between objects and functions.
  - The concept of ***polymorphism*** is often expressed by the phrase "**one interface, multiple methods**."
  - At run-time, the virtual function is matched with the appropriate member function of the object in question

OO Concepts and Terminology (cont.)



OO Concepts and Terminology (cont.)

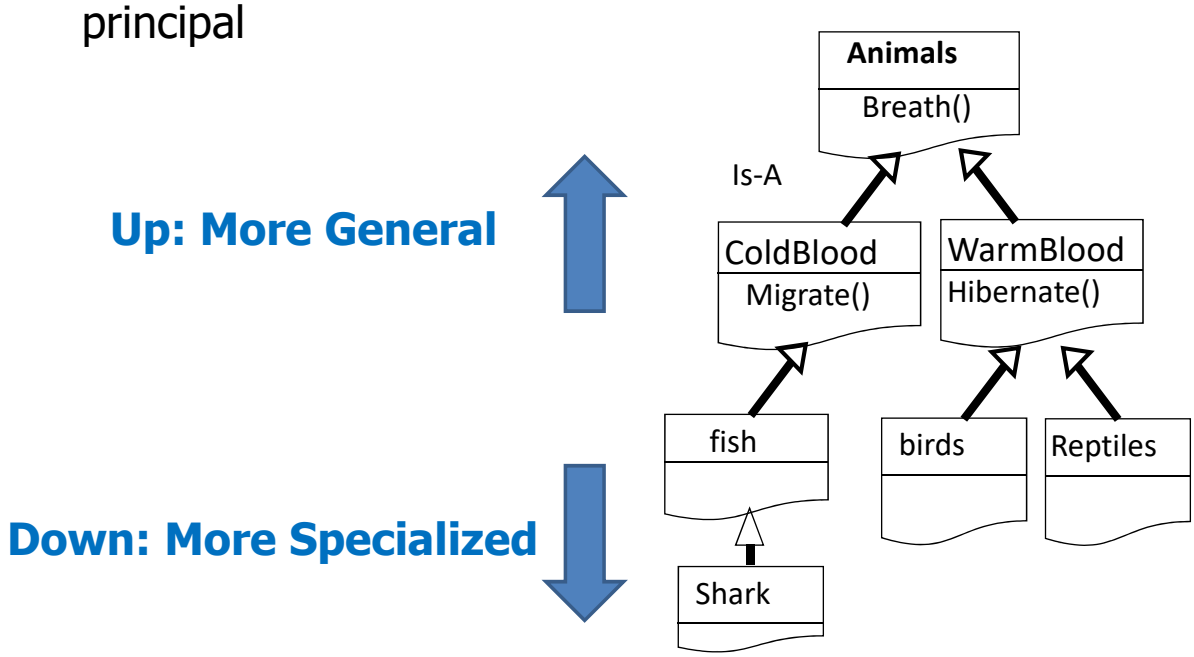
- **Specialization/Generalization Relationship**
  - Specialized element is the **child**
  - Generalized element is the **parent**

  - ***Generalization*** is the process of extracting shared characteristics from two or more classes, and combining them into a generalized superclass.

  - ***Specialization*** means creating new subclasses from an existing class.

Class Hierarchy

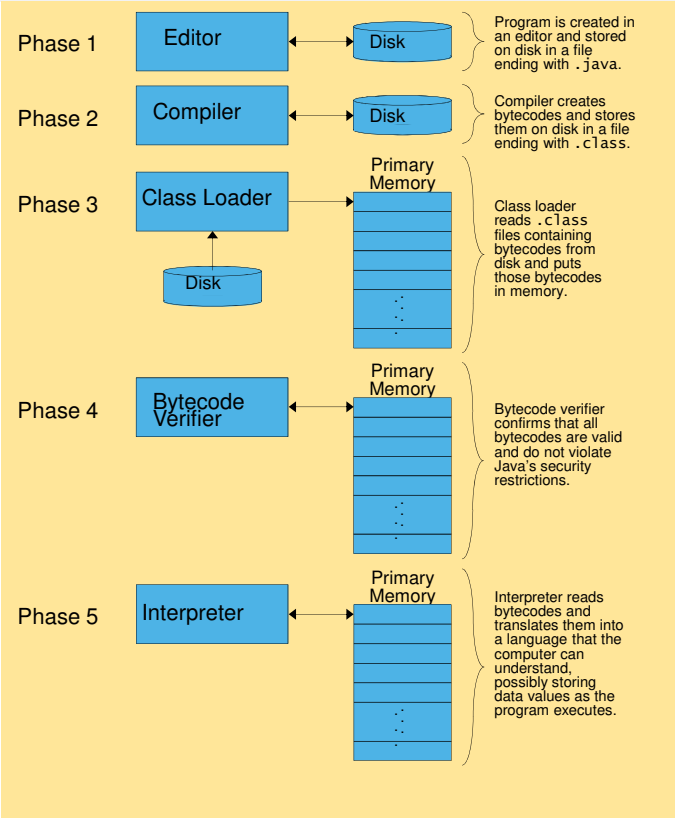
- A set of related classes by generalization-specialization principal



Typical Java Development Environment

- Java programs normally go through five phases:
  1. **Edit**
     - **Programmer** writes **program** (and stores program on disk)
  2. **Compile**
     - **Compiler** creates ***bytecodes*** from program
  3. **Load**
     - Class **loader** stores **bytecodes** in memory
  4. **Verify**
     - **Verifier** ensures **bytecodes** do not violate security requirements
  5. **Execute**
     - **Interpreter** translates **bytecodes** into **machine language**

Typical Java Development Environment (cont.)



A simple java program

```java
// Fig. 2.1: Welcome1.java
// Text-printing program.

public class Welcome1 {

   // main method begins execution of Java application
   public static void main( String args[] )
   {
      System.out.println( "Welcome to Java Programming!" );

   } // end method main

} // end class Welcome1
```

- **Output:**

```
Welcome to Java Programming!
```


A simple java program (cont.)

- You can write a comment in Java like in C++
  - // : Single line comment
  - /* */: multiple lines comment
- Every Java program has at least one user-defined class
- Name of class called identifier
- Java follows the same naming rules as in C++
  - Series of characters consisting of letters, digits, underscores (_) and dollar signs ($)
  - Does not begin with a digit and .has no spaces
- Saving files
  - The **File name** must be the **class name** with `.java` extension
  - `Welcome1.java`

A simple java program (cont.)

- In **Java**, you need to **have** a **method** named **main** in at least one **class**.

- System.out
  - Standard output object

- System.out.println
  - A method that displays a line of text.

Compiling and Executing

- **Compiling a program**
  - Open a command prompt window, go to directory where program is stored
  - Type **javac Welcome1.java**
  - If no errors, Welcome1.class created
    - Has bytecodes that represent application
    - Bytecodes passed to Java interpreter
- **Executing a program**
  - Type **java Welcome1**
    - Interpreter loads .class file for class Welcome1
    - .class extension omitted from command
  - Interpreter calls method main

Running Java File with single command (Java 11)

- One major change in **Java 11** is that you don't need to compile the java source file with `javac` tool first.
- You can directly run the file with `java` command and *it implicitly compiles*.

java Welcome1.java

Modifying Our First Java Program

Escape characters

- Backslash (\ )
- Indicates special characters be output

Escape sequence	Description
`\n`	Newline. Position the screen cursor at the beginning of the next line.
`\t`	Horizontal tab. Move the screen cursor to the next tab stop.
`\r`	Carriage return. Position the screen cursor at the beginning of the current line; do not advance to the next line. Any characters output after the carriage return overwrite the characters previously output on that line.
`\\`	Backslash. Used to print a backslash character.
`\"`	Double quote. Used to print a double -quote character. For exampl e, `System.out.println( "\"in quotes \"" );` displays `"in quotes"`

Fig. 2.5 Some common escape sequences.

Naming Conventions in Java

- Naming conventions for identifiers have been established and **suggested** by various Java communities.

Name	Convention	Example
Classes	Class names should be nouns with the first letter of every word capitalized.	*class Store;* *class FirstClass;*
Methods	Methods should be verbs in lower case or a multi-word name with the first letter lowercase and the first letters of subsequent words in uppercase.	*run();* *insertRecord();* *getBackground();*
Variables	Local variables, instance variables, and class variables are also written in lower case. Variable names should not start with underscore (_) or dollar sign ($) characters, even though both are allowed.	*int i;* *char c;* *float myWidth;*
Constants	Constants should be written in uppercase characters separated by underscores.	*static final int MAX_SIZE = 10;*

Ex: Adding Integers

```
// Fig. 2.7: Addition.java
// Addition program that displays the sum of two numbers.
import java.util.Scanner; // program uses class Scanner

public class Addition
{
   // main method begins execution of Java application
   public static void main( String[] args )
   {
      // create a Scanner to obtain input from the command window
      Scanner input = new Scanner( System.in );

      int number1; // first number to add
      int number2; // second number to add
      int sum; // sum of number1 and number2

      System.out.print( "Enter first integer: " ); // prompt
      number1 = input.nextInt(); // read first number from user

      System.out.print( "Enter second integer: " ); // prompt
      number2 = input.nextInt(); // read second number from user

      sum = number1 + number2; // add numbers, then store total in sum

      System.out.printf( "Sum is %d\n", sum ); // display sum
   } // end method main
} // end class Addition
```

Ex: Adding Integers (cont)

- ***import java.util.Scanner;***
  - Programmers use ***import*** declarations to identify the predefined classes used in a Java program.
  - A great strength of Java is its rich set of predefined classes that you can reuse.
  - These classes are grouped into **packages** (*named collections of related classes*) and are collectively referred to as the Java class library, or the Java Application Interfaces (API)
    - **e.g.** the package java.util contains a variety of utility classes including the collections framework, internationalization, String utilities, etc..

Ex: Adding Integers (cont)

- **System.out:** standard output object that allows Java applications to display characters in the command window.
- **System.in:** standard input object that enables Java applications to read information typed by the user at the keyboard.
- **Scanner**: A simple text scanner which can parse primitive types and strings using regular expressions.
  - A *Scanner* breaks its input into tokens using a delimiter pattern, which by default matches whitespace.
  - The resulting tokens may then be converted into values of different types using the various next methods.
  - Example Methods:
    - **next**()
    - **nextBoolean**()
    - **nextByte** ()
    - **nextDouble**()
    - **nextFloat**()
    - **nextInt**()
    - …..

java.lang Package

- The Java package **java.lang** contains fundamental classes and interfaces closely tied to the language and runtime system.

- This includes the root classes that form the class hierarchy, types tied to the language definition, basic exceptions, math functions, threading, security functions, as well as some information on the underlying native system.

- Classes in **java.lang** are automatically imported into every source file.
  - e.g. Class **System** is part of package **java.lang.**

Java API Documentation

- The package information is important because it helps you locate descriptions of each package and class in the **Java API documentation.**

- **A Web-based version of this documentation can** be found at:

https://docs.oracle.com/javase/8/docs/
https://docs.oracle.com/en/java/javase/11/

- Also, you can download this documentation to your own computer from

http://www.oracle.com/technetwork/java/javase/downloads/index.html#docs

Arithmetic in Java

Java operation	Arithmetic operator	Algebraic expression	Java expression
Addition	+	$f + 7$	f + 7
Subtraction	–	$p - c$	p - c
Multiplication	*	bm	b * m
Division	/	x / y or $\frac{x}{y}$ or $x \div y$	x / y
Remainder	%	$r \bmod s$	r % s

Operator(s)	Operation(s)	Order of evaluation (precedence)
* / %	Multiplication Division Remainder	Evaluated first. If there are several operators of this type, they are evaluated from left to right.
+ -	Addition Subtraction	Evaluated next. If there are several operators of this type, they are evaluated from left to right.

Decision Making: Equality and Relational Operators

- **Equality and relational operators**

Standard algebraic equality or relational operator	Java equality or relational operator	Sample Java condition	Meaning of Java condition
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y

- **Precedence and associativity of operations**

Operators	Associativity	Type
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
=	right to left	assignment

Displaying Text in a Dialog Box

```
1   // Java packages
2   import javax.swing.JOptionPane;  // program uses JOptionPane
3   public class Welcome4 {
4      // main method begins execution of Java application
5     public static void main( String args[] )
6     {
7        JOptionPane.showMessageDialog(
8           null, "Welcome\nto\nJava\nProgramming!" );
9        System.exit( 0 );  // terminate application with window
10    } // end method main
11  } // end class Welcome4
```

- There are types of packages:
  - **Core packages**:- begin with **java.**
  - **Extension packages**: begin with javax.
- You can import one class (e.g. *import javax.swing.JOptionPane;*) or you can import all the classes (e.g. *import javax.swing.*;*)

Displaying Text in a Dialog Box (cont.)

- Class JOptionPane:
  - makes it easy to pop up a standard dialog box that prompts users for a value or informs them of something.
  - allows you to easily create dialog boxes for input and output.
  - `showMessageDialog` is a **`static`** method of class `JOptionPane`
    - **`static`** methods called using class name, dot (**.**) then method name

Displaying Text in a Dialog Box (cont.)

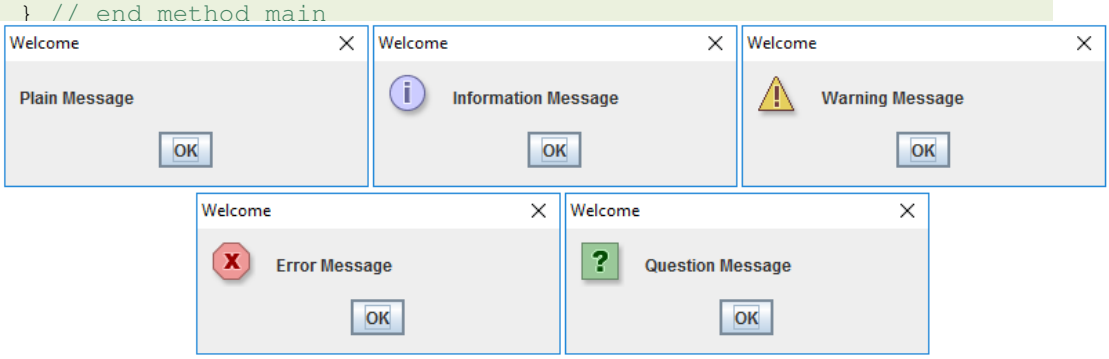
showMessageDialog( ) method can take 2 or 4 arguments:

1. null: to display the message dialog in the center.
2. String: the message to display.
3. String: the string that should appear in the dialog box's title.
4. Message Dialog Type: (Constant)

Message dialog type	Icon	Description
`JOptionPane.ERROR_MESSAGE`		Displays a dialog that indicates an error to the user.
`JOptionPane.INFORMATION_MESSAGE`		Displays a dialog with an informational message to the user. The user can simply dismiss the dialog.
`JOptionPane.WARNING_MESSAGE`		Displays a dialog that warns the user of a potential problem.
`JOptionPane.QUESTION_MESSAGE`		Displays a dialog that poses a question to the user. This dialog normally requires a response, such as clicking on a **Yes** or a **No** button.
`JOptionPane.PLAIN_MESSAGE`	no icon	Displays a dialog that simply contains a message, with no icon.

Displaying Text in a Dialog Box (cont.)

```
import javax.swing.JOptionPane;
public class MessageDialogTest {
public static void main(String[] args)
   {
      JOptionPane.showMessageDialog( null, "Plain Message" ,
      "Welcome", JOptionPane.PLAIN_MESSAGE );
      JOptionPane.showMessageDialog( null, "Information Message" ,
      "Welcome", JOptionPane.INFORMATION_MESSAGE );
      JOptionPane.showMessageDialog( null, "Error Message" ,
      "Welcome", JOptionPane.ERROR_MESSAGE );
      JOptionPane.showMessageDialog( null, "Warning Message" ,
      "Welcome", JOptionPane.WARNING_MESSAGE );
      JOptionPane.showMessageDialog( null, "Question Message" ,
          "Welcome", JOptionPane.QUESTION_MESSAGE);
   } // end method main
}
```



Input Dialog Box

```
import javax.swing.JOptionPane;

public class NameDialog
{
   public static void main( String[] args )
   {
      // prompt user to enter name
      String name =
         JOptionPane.showInputDialog( "What is your name?" );

      // create the message
      String message =
         String.format( "Welcome, %s, to Java Programming!", name );

      // display the message to welcome the user by name
      JOptionPane.showMessageDialog( null, message );
   } // end main
} // end class NameDialog
```

- String class is located in java.lang

Primitive Types vs. Reference Types

- Java has two types: **primitive** and **reference**
- **Primitives types**
  - boolean: boolean (*Note : unlike in C++, boolean is different than int*)
    - Its values are either **true** or **false**
  - Integer
    - int, long, byte, short, char.
  - Floating
    - float, double
- **Reference types**
  - Class types, Interface types, and Arrays

    e.g. *Calculator c;*

    int [] A;

Java Primitive Data types

- All variables must first be declared before they can be used.
- Primitive types are special data types built into the language
- There are 8 primitive types:
  - **byte**: The byte data type is an 8-bit signed two's complement integer. The byte data type can be useful for saving memory in large arrays, where the memory savings actually matters.
  - **short**: The short data type is a 16-bit signed two's complement integer.
  - **int**: The int data type is a 32-bit signed two's complement integer.
  - **long**: The long data type is a 64-bit signed two's complement integer. Use this data type when you need a range of values wider than those provided by int.
  - **float**: The float data type is a single-precision 32-bit IEEE 754 floating point.
  - **double**: The double data type is a double-precision 64-bit IEEE 754 floating point.
  - **boolean**: The boolean data type has only two possible values: true and false.
  - **char**: The char data type is a single 16-bit Unicode character.

Default Values

- It's not always necessary to assign a value when a field is declared.
- Fields that are declared but not initialized will be set to a reasonable default by the compiler.

Data Type	Default Value
byte	**0**
short	**0**
int	**0**
long	**0L**
float	**0.0f**
double	**0.0d**
char	**'\u0000'**
String (or any object)	**null**
boolean	**false**

Literals

- A *literal* is the source code representation of a fixed value;
- It is possible to assign a literal to a variable of a primitive type:

```
boolean result = true;
char    capitalC = 'C';
char    a = '\u0108' ; // Unicode escape,
Byte    b = 100;
short   s = 10000;
int     i = 100000;
int     decVal = 26; // The number 26, in decimal
int     octVal = 032; // The number 26, in octal
int     hexVal = 0x1a; // The number 26, in hexadecimal
double  d1 = 123.4;
double  d2 = 1.234e2; // same value as d1, but in scientific notation
float   f1 = 123.4f;
```


Java Keywords

abstract	continue	for	new	switch
assert	default	goto*	package	synchronized
boolean	do	if	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const*	float	native	super	while

** Not used*

Creating and accessing Reference Variables

- Creating a reference variable
  1. Declaration

     Calculator c;
  2. Initialization

     c = new Calculator( ); // calling the constructor of the class calculator
- Accessing reference variables
  – Just like accessing instance of objects in C++

    c.add(5,6); // add is a method in Class Calculator

Wrapper Classes

- Wrapper classes are classes that are used to manipulate primitive type values as objects.
- Wrapper classes are useful in storing primitive data types in higher level data structures such as Stack<Object>.
- There are **8 wrapper classes** provided in **java.lang** for the corresponding primitive types
- Primitive types do not have methods, so the methods related to a primitive type are located in the corresponding type-wrapper class.

Primitive type	Wrapper class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Ex: Adding Integers

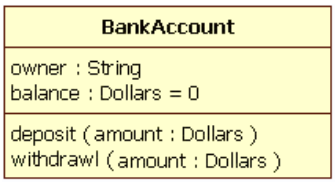
```
1   import javax.swing.JOptionPane;  // program uses JOptionPane
2   public class Addition {
3   public static void main( String args[] )
4   {
5           String firstNumber;   // first string entered by user
6           String secondNumber;  // second string entered by user
7           int number1;          // first number to add
8           int number2;          // second number to add
9           int sum;              // sum of number1 and number2
10           // read in first number from user as a String
11          firstNumber = JOptionPane.showInputDialog( "Enter first integer" );
12          // read in second number from user as a String
13          secondNumber =
14          JOptionPane.showInputDialog( "Enter second integer" );
15          // convert numbers from type String to type int
16          number1 = Integer.parseInt( firstNumber );
17          number2 = Integer.parseInt( secondNumber );
18           // add numbers
19          sum = number1 + number2;
20           // display result
21          JOptionPane.showMessageDialog( null, "The sum is " + sum,
22           "Results", JOptionPane.PLAIN_MESSAGE );
23          System.exit( 0 );   // terminate application with window
24         } // end method main
25  } // end class Addition
```

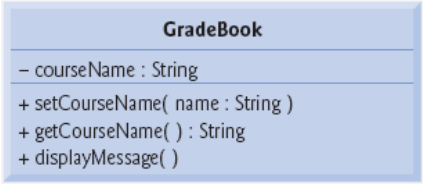

Autoboxing and Auto-Unboxing

- A boxing conversion converts a value of a primitive type to an object of the corresponding type-wrapper class.
- An unboxing conversion converts an object of a type-wrapper class to a value of the corresponding primitive type.
- These conversions can be performed automatically (called autoboxing and auto-unboxing).
- Example:
  -
```
// create integerArray
Integer[] integerArray = new Integer[ 5 ];

// assign Integer 10 to integerArray[ 0 ]
integerArray[ 0 ] = 10;

// get int value of Integer
int value = integerArray[ 0 ];
```

Unified Modeling Language (UML)

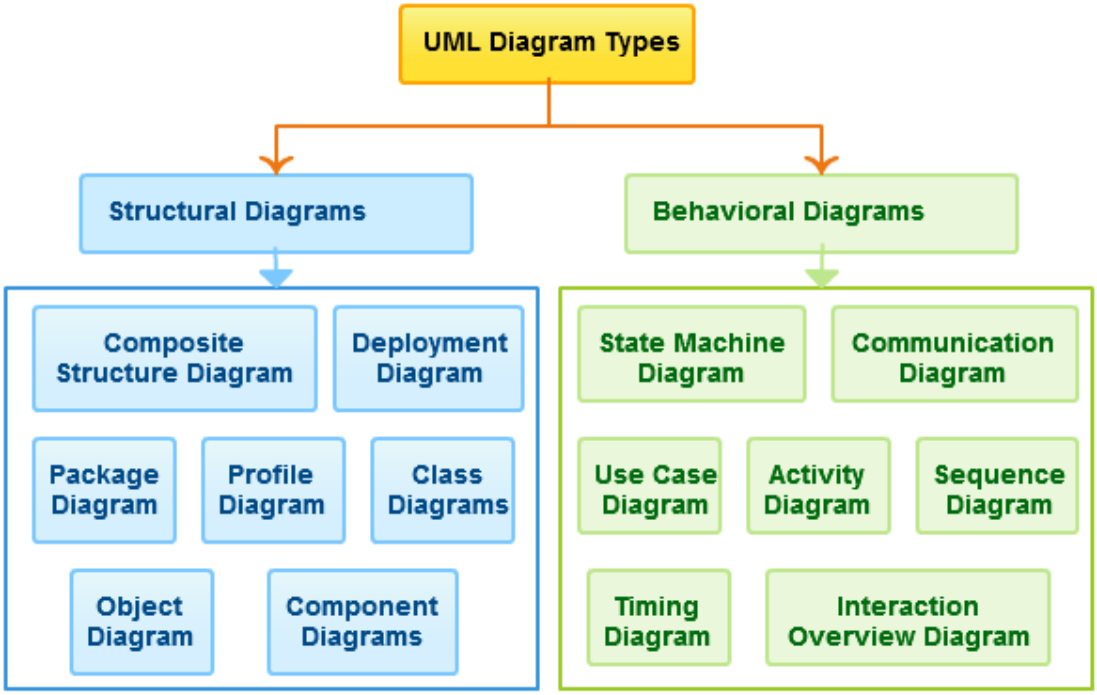
- A graphical language that allows people who design software systems to use an industry standard notation to represent them.
- Allows developers to represent object-oriented designs
- Was developed in the mid-1990s
- The UML is now the most widely used graphical representation scheme for modeling OO Systems.
- UML 2.4 has 14 types of diagrams divided into two categories. Seven diagram types represent *structural* information, and the other seven represent general types of *behavior*.
  - e.g. class diagram





See: *www.uml.org*
http://en.wikipedia.org/wiki/Unified_Modeling_Language

UML Diagram Types



Instance Variables, *set Methods and get Methods*

```
public class GradeBook
{
        private String courseName; // course name for this GradeBook
        // method to set the course name
        public void setCourseName( String name )
        {
                courseName = name; // store the course name
        } // end method setCourseName

        // method to retrieve the course name
        public String getCourseName()
        {
                return courseName;
        } // end method getCourseName

        // display a welcome message to the GradeBook user
        public void displayMessage()
        {
                // this statement calls getCourseName to get the
                // name of the course this GradeBook represents
                System.out.printf( "Welcome to the grade bookfor\n%s!\n",
                                getCourseName());
        } // end method displayMessage

} // end class GradeBook
```


Instance Variables, *set Methods and get Methods* (cont.)

```
import java.util.Scanner; // program uses Scanner
public class GradeBookTest
 {
         // main method begins program execution
        public static void main( String args[] )
        {
                // create Scanner to obtain input from command window
                Scanner input = new Scanner( System.in );
                 // create a GradeBook object and assign it to myGradeBook
                 GradeBook myGradeBook = new GradeBook();

                 // display initial value of courseName
                System.out.printf( "Initial course name is: %s\n\n",
                        myGradeBook.getCourseName() );

                 // prompt for and read course name
                System.out.println( "Please enter the course name:" );
                String theName = input.nextLine(); // read a line of text
                myGradeBook.setCourseName( theName ); // set the course name
                System.out.println(); // outputs a blank line
                // display welcome message after specifying course name
                myGradeBook.displayMessage();

         } // end main
 } // end class GradeBookTest
```

Access Modifiers in JAVA

- **Level access modifiers (java classes only)**
  - Only two access modifiers is allowed, ***public*** *and* ***no modifier***
  - If a class is '**public**', then it **CAN** be accessed from **ANYWHERE**.
  - If a class has '**no modifer**', then it **CAN ONLY** be accessed from '**same package**'.
- **Member level access modifiers (java variables and java methods)**
  - All the four ***public***, ***private***, ***protected*** *and* ***no modifer*** is allowed.
  - ***public*** and ***no modifier*** – the same way as used in class level.
  - ***private*** – members CAN ONLY access.
  - ***protected*** – CAN be accessed from **'same package'** and a **subclass** existing in any package can access.

Access Modifiers	*Same Class*	*Same Package*	*Subclass*	*Other packages*
Public	Y	Y	Y	Y
protected	Y	Y	Y	N
no access modifier	Y	Y	N	N
private	Y	N	N	N

Control Statements [as in C/C++]

- **if** statement
- **if-else** statement
- **switch** statement
- **for** loop statement
- **while** loop statament
- **do –while** loop statement
- **break** statement
- **continue** statement
- Conditional Operator **? :**
- Nested loops
- Logical and condition operators

Increment and Decrement Operators

Operator	Operator name	Sample expression	Explanation
++	prefix increment	++a	Increment a by 1, then use the new value of a in the expression in which a resides.
++	postfix increment	a++	Use the current value of a in the expression in which a resides, then increment a by 1.
--	prefix decrement	--b	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	postfix decrement	b--	Use the current value of b in the expression in which b resides, then decrement b by 1.

Precedence/Associativity of the operators

Operators	Associativity	Type
++ --	right to left	unary postfix
++ - + - ! (*type*)	right to left	unary prefix
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&	left to right	boolean logical AND
^	left to right	boolean logical exclusive OR
|	left to right	boolean logical inclusive OR
&&	left to right	conditional AND
||	left to right	conditional OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

static Methods, static Fields

- Every class provides methods that perform common tasks on objects of the class.
- Sometimes a method performs a task that does not depend on the contents of any object.
- Such a method applies to the class in which it is declared as a whole and is known as a **static method** or a **class method.**
- To declare a method as static, place the keyword **static** before the return type in the method's declaration.
- You can call any static method by specifying the name of the class in which the method is declared, followed by a dot (.) and the method name

Math-Class Methods

- Class `java.lang.Math`
  - provides a collection of methods that enable you to perform common mathematical calculations.

Method	Description	Example

`abs( x )`	absolute value of *x* (this method also has `float`, `int` and `long` versions)	`abs( 23.7 )` is `23.7`
		`abs( 0.0 )` is `0.0`
		`abs( -23.7 )` is `23.7`

`ceil( x )`	rounds *x* to the smallest integer not less than *x*	`ceil( 9.2 )` is `10.0` `ceil( -9.8 )` is `-9.0`
`cos( x )`	trigonometric cosine of *x* (*x* is in radians)	`cos( 0.0 )` is `1.0`
`exp( x )`	exponential method *ex*	`exp( 1.0 )` is `2.71828` `exp( 2.0 )` is `7.38906`
`floor( x )`	rounds *x* to the largest integer not greater than *x*	`floor( 9.2 )` is `9.0` `floor( -9.8 )` is `-10.0`
`log( x )`	natural logarithm of *x* (base *e*)	`log( Math.E )` is `1.0` `log( Math.E * Math.E )` is `2.0`
`max( x, y )`	larger value of *x* and *y* (this method also has `float`, `int` and `long` versions)	`max( 2.3, 12.7 )` is `12.7` `max( -2.3, -12.7 )` is `-2.3`
`min( x, y )`	smaller value of *x* and *y* (this method also has `float`, `int` and `long` versions)	`min( 2.3, 12.7 )` is `2.3` `min( -2.3, -12.7 )` is `-12.7`
`pow( x, y )`	*x* raised to the power *y* (*xy*)	`pow( 2.0, 7.0 )` is `128.0` `pow( 9.0, 0.5 )` is `3.0`
`sin( x )`	trigonometric sine of *x* (*x* is in radians)	`sin( 0.0 )` is `0.0`
`sqrt( x )`	square root of *x*	`sqrt( 900.0 )` is `30.0` `sqrt( 9.0 )` is `3.0`
`tan( x )`	trigonometric tangent of *x* (*x* is in radians)	`tan( 0.0 )` is `0.0`

Constant fields

- Class Math also declares two fields that represent commonly used mathematical constants:
  - **Math.PI** (3.14159265358979323846)
  - Math.E (2.7182818284590452354) is the base value for natural logarithms
- These fields are declared in class Math with the modifiers public, final and static.
  - public keyword: allows other programmers to use these fields in their own classes
  - Final keyword: Any field declared with keyword final is constant—its value cannot be changed after the field is initialized.
  - static: Allows them to be accessed via the class name

Argument Promotion

- argument promotion—converting an argument's value to the type that the method expects to receive in its corresponding parameter.
  - e.g. a program can call Math method sqrt with an **integer** argument even though the method expects to receive a **double** argument
- **Promotion rules**
  - Specify how to convert types without data loss

Type	Valid promotions
double	None
float	double
long	float or double
int	long, float or double
char	int, long, float or double
short	int, long, float or double
byte	short, int, long, float or double
boolean	None (boolean values are not considered to be numbers in Java)

JAVA API Packages

Package	Description
`java.applet`	The ***Java Applet Package*** contains the Applet class and several interfaces that enable applet/browser interaction and the playing of audio clips. In Java 2, class javax.swing.JApplet is used to define an applet that uses the Swing GUI components.
`java.awt`	The ***Java Abstract Window Toolkit*** *Package* contains the classes and interfaces required to create and manipulate GUIs in Java 1.0 and 1.1. In Java 2, the Swing GUI components of the javax.swing packages are often used instead.
`java.awt.event`	The ***Java Abstract Window Toolkit Event*** *Package* contains classes and interfaces that enable event handling for GUI components in both the java.awt and javax.swing packages.
`java.io`	The ***Java Input/Output*** *Package* contains classes that enable programs to input and output data
`java.lang`	The ***Java Language*** *Package* contains classes and interfaces (discussed throughout this text) that are required by many Java programs. This package is imported by the compiler into all programs.
`java.net`	The ***Java Networking*** *Package* contains classes that enable programs to communicate via networks
`java.text`	The ***Java Text*** *Package* contains classes and interfaces that enable a Java program to manipulate numbers, dates, characters and strings. The package provides many of Java's internationalization capabilities that enable a program to be customized to a specific locale (e.g., an applet may display strings in different languages, based on the user's country).
`java.util`	The ***Java Utilities*** *Package* contains utility classes and interfaces, such as date and time manipulations, random-number processing capabilities with class Random, storing and processing large amounts of data and breaking strings into smaller pieces called *tokens* with class String-Tokenizer
`javax.swing`	The ***Java Swing GUI Components*** *Package* contains classes and interfaces for Java's Swing GUI components that provide support for portable GUIs.
`javax.swing.event`	The ***Java Swing Event Package*** contains classes and interfaces that enable event handling for GUI components in package javax.swing.

Random-Number Generation

- There are two options to generate a random number stream.
  - **Math** class provides a static method random() that returns a double value with a positive sign, greater than or equal to 0.0 and less than 1.0

    Ex: *To generate and int number in a specific range*

    ```
    int x;
    x =  ( int ) ( Math.random() * 6 );//Produces integers from 0 - 5
    ```

  - Class **Random** (package java.util) provides a set of methods
    - `boolean nextBoolean();`
    - `double nextDouble();`
    - `float nextFloat();`
    - `int nextInt();`
    - `int nextInt(int n);`

    (you have to create an object from this class in order to call these methods)

Random-Number Generation (cont.)

```
import java.util.Random;
public class RandomIntegers
{
      public static void main( String args[] )
      {
         Random randNumbers = new Random();
         int face; // stores each random integer generated

         // loop 20 times
         for ( int counter = 1; counter <= 20; counter++ ){
                  // pick random integer from 1 to 6
             face = 1 + randNumbers.nextInt(6);
             System.out.printf( "%d ", face ); // display generated value
             // if counter is divisible by 5, start a new line of output
             if ( counter % 5 == 0 )
                 System.out.println();
         } // end for
    } // end main
 } // end class RandomIntegers
```

```
5 3 2 3 2
5 2 6 2 6
2 5 5 1 4
2 3 6 4 4
```


Random-Number Generation (cont.)

```
public class RandomIntegers
{
        public static void main( String args[] )
        {
            int face; // stores each random integer generated
            // loop 20 times
            for ( int counter = 1; counter <= 20; counter++ ){
                // pick random integer from 1 to 6
                face = 1 + (int)(6*Math.random());
                System.out.printf( "%d ", face ); // display generated value
                // if counter is divisible by 5, start a new line of output
                if ( counter % 5 == 0 )
                        System.out.println();
            } // end for
        } // end main
 } // end class RandomIntegers
```

Static import

- A ***static import*** declaration enables you to import **static** members of a class so you can access them via their ***unqualified*** names in your class.
- Two forms:
  - One that imports a particular ***static*** member:

```
import static  java.lang.Math.PI;
```

  - One that import all ***static*** members

```
import static  java.lang.Math.*;
```


Static import (cont.)

```
import static java.lang.Math.*;

public class StaticImportTest {
   public static void main(String[] args) {
      System.out.printf("sqrt(900.0) = %.1f%n", sqrt(900.0));
      System.out.printf("ceil(-9.8) = %.1f%n", ceil(-9.8));
      System.out.printf("E = %f%n", E);
      System.out.printf("PI = %f%n", PI);
   }
}
```

Arrays

- Arrays are objects, so they are considered reference types.
- An *array* is a container object that holds a fixed number of values of a single type.
- The length of an array is established when the array is created.
- After creation, its length is fixed.
- **Declaring** a Variable to Refer to an Array:

```
int  anArray []; // declares an array of integers
```

- **Creating** an Array:

```
anArray = new int[10]; // create an array of integers
```

- You can combine the previous two steps:

```
int anArray[] = new int[10];
```

- You can use the shortcut syntax to **create and initialize** an array:

```
int anArray[] = {100, 200, 300, 400, 600, 900, 1000};
// Using an Array Initializer without new
```


Arrays (cont.)

- When an array is declared, the type of the array and the square brackets can be combined at the beginning of the declaration to indicate that all the identifiers in the declaration are array variables.

```
double[] array1, array2;
```

➔ **the square brackets can be placed either after the type or after the array variable name.**

		a	b
`int a[] , b[];`	valid	Array	Array
`int []a , b ;`	valid	Array	Array
`int a , b[];`	valid	variable	Array
`int a , []b;`	invalid	-	-

Arrays (cont.)

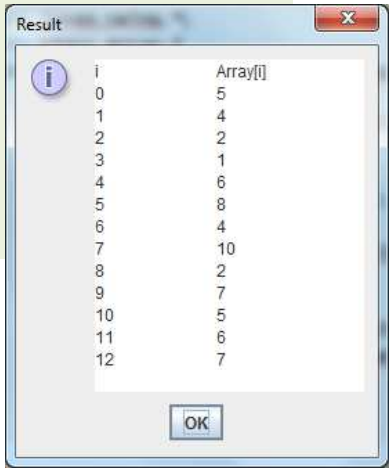
- Length
  - **length** is a member variable of any array
  - **(.length)** gives the length of an array

- When looping through an array
  - Index should never go below 0
  - Index should be less than the total number of array elements

- When invalid array reference occurs
  - Java generates ArrayIndexOutOfBoundsException

Arrays (cont.)

```
import javax.swing.*;
public class ArrayTest {
  public static void main(String[] args)
  {
      int array[] = {5,4,2,1,6,8,4,10,2,7,5,6,7}; //do not use new here
      String out = "i\tArray[i]\n";
      JTextArea t1 = new JTextArea();
      for(int i =0 ;i<array.length;i++)
      {
              out += i +"\t" + array[i] +"\n";
      }
      t1.setText(out);
      JOptionPane.showMessageDialog(null,t1,"Result",
              JOptionPane.INFORMATION_MESSAGE);
      System.exit(0);
  }
}
```



Enhanced for Statement

- The **enhanced for statement iterates through the elements of an array or a collection without using a** counter
- The syntax of an enhanced for statement is:

```
for ( parameter : arrayName )
        statement
```

```
class EnhancedForDemo {
    public static void main(String[] args){
            int[] numbers = {1,2,3,4,5,6,7,8,9,10};
            int sum = 0;
            for (int item : numbers) {
                    sum += item;
            }
            System.out.println("Sum =" + sum);
    }
}
```


Reference Parameters

- Two ways to pass arguments to methods
  - Pass-by-value
    - Copy of argument's value is passed to called method
    - In Java, every primitive is pass-by-value
  - Pass-by-reference
    - Caller gives called method direct access to caller's data
    - Called method can manipulate this data
    - Improved performance over pass-by-value
    - In Java, every object is pass-by-reference
      - In Java, arrays are objects

      ➔Therefore, arrays are passed to methods by reference

Passing Arrays to Methods

- For a method to receive an array reference through a method call, the method's parameter list must specify an array parameter.

```
void modifyArray( double b[] )
```

- To pass an array argument to a method, specify the name of the array without any brackets.

```
double hourlyTemperatures[] = new double[ 24 ];
modifyArray( hourlyTemperatures );
```

- When the called method uses the array variable, it refers to the same array object as in the caller.

Copying Arrays

- The System class has an arraycopy() method that you can use to efficiently copy data from one array into another:

```
public static void arraycopy(Object src, int srcPos,
                             Object dest, int destPos, int length)
```

```
1   class ArrayCopyDemo {
2      public static void main(String[] args) {
3          char[] copyFrom = { 'd', 'e', 'c', 'a', 'f', 'f', 'e',
4                              'i', 'n', 'a', 't', 'e', 'd' };
5          char[] copyTo = new char[7];
6                                  من وين
                                   يبدا النسخ
                                                من وين يبدا النسخ
                                                للعنصر الجديد
7          System.arraycopy(copyFrom, 2, copyTo, 0, 7);
8          System.out.println(new String(copyTo));
                                                    عدد العناصر المنسوخة
9      }
10 }
```

- The output from this program is: caffein

Multidimensional Arrays

- Multidimensional arrays
  - Tables with rows and columns
    - Two-dimensional array
    - Declaring two-dimensional array b[2][2]

```
int b[][] = { { 1, 2 }, { 3, 4 } };
```

      - 1 and 2 initialize b[0][0] and b[0][1]
      - 3 and 4 initialize b[1][0] and b[1][1]
    - Two-dimensional array with Rows of Different Lengths

```
int b[][] = { { 1, 2 }, { 3, 4, 5 } };
```

      - row 0 contains elements 1 and 2
      - row 1 contains elements 3, 4 and 5

Multidimensional Arrays (cont.)

- Creating multidimensional arrays
  - Can be allocated dynamically
    - 3-by-4 array

```
int b[][];
b = new int[ 3 ][ 4 ];
```

    - Rows can have different number of columns

```
int b[][];
b = new int[ 2 ][ ];    // allocate rows
b[ 0 ] = new int[ 5 ]; // allocate row 0
b[ 1 ] = new int[ 3 ]; // allocate row 1
```

Multidimensional Arrays (cont.)

```
class TwoDimArray{
   public static void main(String args[]) {
   int twoD[][] = new int[4][];
       twoD[0] = new int[1];
       twoD[1] = new int[2];
       twoD[2] = new int[3];
       twoD[3] = new int[4];

       int i, j , k = 0;

       for(i = 0; i < 4; i++)
                for(j = 0; j < i+1; j++){
                         twoD[i][j] = k;
                         k++;
                }

       for(i = 0; i < 4; i++){
                for(j = 0; j < i+1; j++)
                         System.out.print(twoD[i][j] + " ");
                         System.out.println();
       }
   }
}
```

```
0
1 2
3 4 5
6 7 8 9
```


Variable-Length Argument Lists

- With **variable-length argument lists, you can create methods that receive an unspecified** number of arguments.
- An argument type followed by an **ellipsis (...) in a method's parameter list indicates that the method receives a variable number of arguments of that particular** type.
- This use of the ellipsis can occur only once in a parameter list, and the ellipsis, together with its type, must be placed at the end of the parameter list.
- The enhanced for statement can be used to manipulate the different arguments.

Variable-Length Argument Lists (cont.)

```java
public class VarargsTest
{
    public static double average( double... numbers ){
        double total = 0.0; // initialize total
        // calculate total using the enhanced for statement
        for ( double d : numbers )
        total += d;
        return total / numbers.length;
    } // end method average

   public static void main( String args[] ) {
        double d1 = 10.0;
        double d2 = 20.0;
        double d3 = 30.0;
        double d4 = 40.0;
        System.out.printf( "d1 = %.1f\nd2 = %.1f\nd3 = %.1f\nd4 = %.1f\n\n",
                d1, d2, d3, d4 );
        System.out.printf( "Average of d1 and d2 is %.1f\n", average( d1, d2 ) );
        System.out.printf( "Average of d1, d2 and d3 is %.1f\n",
                average( d1, d2, d3));
        System.out.printf( "Average of d1, d2, d3 and d4 is %.1f\n",
                average( d1, d2, d3, d4 ) );
    } // end main
} // end class VarargsTest
```

```
d1 = 10.0
d2 = 20.0
d3 = 30.0
d4 = 40.0

Average of d1 and d2 is 15.0
Average of d1, d2 and d3 is 20.0
Average of d1, d2, d3 and d4 is 25.0
```


Using Command-Line Arguments

- Sometimes you will want to pass information into a program when you run it.
- This is accomplished by passing command-line arguments to main( ).
- A command-line argument is the information that directly follows the program's name on the command line when it is executed.

```
c:\ javac Test.java
c:\ java  Test  Hello  1234
```

- To access the command-line arguments inside a Java program is quite easy.
  - they are stored as strings in the **String array** passed to main( ).

```java
public static void main( String args[] )
```

Using Command-Line Arguments (cont.)

```java
// Fig. 7.21: InitArray.java
// Using command-line arguments to initialize an array.

public class InitArray
{
   public static void main( String args[] )
   {
      // check number of command-line arguments
      if ( args.length != 3 )
         System.out.println(
            "Error: Please re-enter the entire command, including\n" +
            "an array size, initial value and increment." );
      else
      {
         // get array size from first command-line argument
         int arrayLength = Integer.parseInt( args[ 0 ] );
         int array[] = new int[ arrayLength ]; // create array

         // get initial value and increment from command-line arguments
         int initialValue = Integer.parseInt( args[ 1 ] );
         int increment = Integer.parseInt( args[ 2 ] );

         // calculate value for each array element
         for ( int counter = 0; counter < array.length; counter++ )
            array[ counter ] = initialValue + increment * counter;

         System.out.printf( "%s%8s\n", "Index", "Value" );

         // display array index and value
         for ( int counter = 0; counter < array.length; counter++ )
            System.out.printf( "%5d%8d\n", counter, array[ counter ] );
      } // end else
   } // end main
} // end class InitArray
```

```
C:\ java InitArray 5 10 15
```

```
Index   Value
    0      10
    1      25
    2      40
    3      55
    4      70
```


Class Arrays

```
import java.util.Arrays
```

- **Arrays** class
  - Provides `static` methods for common array manipulations.
- Methods include
  - ***sort*** for sorting an array (ascending order by default)
  - ***binarySearch*** for searching a sorted array
  - ***equals*** for comparing arrays
  - ***fill*** for placing values into an array.
- Methods are overloaded for primitive-type arrays and for arrays of objects.

Class Arrays (cont.)

```
import java.util.Arrays;

public class ArrayManipulations
{
   public static void main(String[] args)
   {
      // sort doubleArray into ascending order
      double[] doubleArray = {8.4, 9.3, 0.2, 7.9, 3.4};
      Arrays.sort(doubleArray);
      System.out.printf("%ndoubleArray: ");

      for (double value : doubleArray)
         System.out.printf("%.1f ", value);

      // fill 10-element array with 7s
      int[] filledIntArray = new int[10];
      Arrays.fill(filledIntArray, 7);
      displayArray(filledIntArray, "filledIntArray");

      // copy array intArray into array intArrayCopy
      int[] intArray = {1, 2, 3, 4, 5, 6};
      int[] intArrayCopy = new int[intArray.length];
      System.arraycopy(intArray, 0, intArrayCopy, 0, intArray.length);
      displayArray(intArray, "intArray");
      displayArray(intArrayCopy, "intArrayCopy");
```


Class Arrays (cont.)

```
      // compare intArray and intArrayCopy for equality
      boolean b = Arrays.equals(intArray, intArrayCopy);
      System.out.printf("%n%nintArray %s intArrayCopy%n",
         (b ? "==" : "!="));
      // compare intArray and filledIntArray for equality
      b = Arrays.equals(intArray, filledIntArray);
      System.out.printf("intArray %s filledIntArray%n",
         (b ? "==" : "!="));
      // search intArray for the value 5
      int location = Arrays.binarySearch(intArray, 5);

      if (location >= 0)
         System.out.printf(
            "Found 5 at element %d in intArray%n", location);
      else
         System.out.println("5 not found in intArray");
      // search intArray for the value 8763
      location = Arrays.binarySearch(intArray, 8763);

      if (location >= 0)
         System.out.printf(
            "Found 8763 at element %d in intArray%n", location);
      else
         System.out.println("8763 not found in intArray");
   }
```

Class Arrays (cont.)

```
   // output values in each array
   public static void displayArray(int[] array, String description)
   {
      System.out.printf("%n%s: ", description);

      for (int value : array)
         System.out.printf("%d ", value);
   }
} // end class ArrayManipulations
```

```
doubleArray: 0.2 3.4 7.9 8.4 9.3
filledIntArray: 7 7 7 7 7 7 7 7 7 7
intArray: 1 2 3 4 5 6
intArrayCopy: 1 2 3 4 5 6

intArray == intArrayCopy
intArray != filledIntArray
Found 5 at element 4 in intArray
8763 not found in intArray
```

Common Programming Error 7.7

Passing an unsorted array to `binarySearch` *is a logic error—the value returned is undefined.*

Generic Methods

- Overloaded methods are often used to perform similar operations on different types of data.
- Detect type mismatches at compile time known as *compile-time type safety.*
- **Generic methods** and **generic classes** provide the means to create type safe general model.
- The term ***generic*** means ***parameterized types***.
  - Enable you to create classes, interfaces, and methods in which the type of data upon which they operate is specified as a parameter.

A WARNING to C++ Programmers

- Although generics are **similar** to templates in C++, **they are not the same**. There are some fundamental differences between the two approaches to generic types.

- If you have a background in C++, it is important not to jump to conclusions about how generics work in Java

Generic Methods (cont.)

```java
// Fig. 20.1: OverloadedMethods.java
// Printing array elements using overloaded methods.

public class OverloadedMethods
{
   public static void main(String[] args)
   {
      // create arrays of Integer, Double and Character
      Integer[] integerArray = {1, 2, 3, 4, 5, 6};
      Double[] doubleArray = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
      Character[] characterArray = {'H', 'E', 'L', 'L', 'O'};

      System.out.printf("Array integerArray contains:%n");
      printArray(integerArray); // pass an Integer array
      System.out.printf("%nArray doubleArray contains:%n");
      printArray(doubleArray); // pass a Double array
      System.out.printg("%nArray characterArray contains:%n");
      printArray(characterArray); // pass a Character array
   }

```

Fig. 20.1 | Printing array elements using overloaded methods. (Part 1 of 3.)

Generic Methods (cont.)

```java
    // method printArray to print Integer array
    public static void printArray(Integer[] inputArray)
    {
       // display array elements
       for (Integer element : inputArray)
          System.out.printf("%s ", element);

       System.out.println();
    }

    // method printArray to print Double array
    public static void printArray(Double[] inputArray)
    {
       // display array elements
       for (Double element : inputArray)
          System.out.printf("%s ", element);

       System.out.println();
    }

```

Fig. 20.1 | Printing array elements using overloaded methods. (Part 2 of 3.)

Generic Methods (cont.)

```java
    // method printArray to print Character array
    public static void printArray(Character[] inputArray)
    {
       // display array elements
       for (Character element : inputArray)
          System.out.printf("%s ", element);

       System.out.println();
    }
 } // end class OverloadedMethods
```

```
Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O
```

Fig. 20.1 | Printing array elements using overloaded methods. (Part 3 of 3.)

Generic Methods (cont.)

- If the operations performed by several overloaded methods are identical for each argument type, the overloaded methods can be more conveniently coded using a generic method.
- You can write a single generic method declaration that can be called with arguments of different types.
- Based on the types of the arguments passed to the generic method, the compiler handles each method call appropriately.

Generic Methods (cont.)

```java
// Fig. 20.3: GenericMethodTest.java
// Printing array elements using generic method printArray.

public class GenericMethodTest
{
   public static void main(String[] args)
   {
      // create arrays of Integer, Double and Character
      Integer[] intArray = {1, 2, 3, 4, 5};
      Double[] doubleArray = {1.1, 2.2, 3.3, 4.4, 5.5, 6.6, 7.7};
      Character[] charArray = {'H', 'E', 'L', 'L', 'O'};

      System.out.printf("Array integerArray contains:%n");
      printArray(integerArray); // pass an Integer array
      System.out.printf("%nArray doubleArray contains:%n");
      printArray(doubleArray); // pass a Double array
      System.out.printf("%nArray characterArray contains:%n");
      printArray(characterArray); // pass a Character array
   }

```

Fig. 20.3 | Printing array elements using generic method **printArray**. (Part 1 of 2.)

Generic Methods (cont.)

```java
    // generic method printArray
    public static <T> void printArray(T[] inputArray)
    {
       // display array elements
       for (T element : inputArray)
          System.out.printf("%s ", element);

       System.out.println();
    }
 } // end class GenericMethodTest
```

```
Array integerArray contains:
1 2 3 4 5 6

Array doubleArray contains:
1.1 2.2 3.3 4.4 5.5 6.6 7.7

Array characterArray contains:
H E L L O
```

Fig. 20.3 | Printing array elements using generic method **printArray**. (Part 2 of 2.)

Generic Methods (cont.)

- All generic method declarations have a type-parameter section (< T > in this example) delimited by angle brackets that precedes the method's return type.
- Each type-parameter section contains one or more type parameters, separated by commas.
- A type parameter, also known as a type variable, is an identifier that specifies a generic type name.
  - Can be used to declare the return type, parameter types and local variable types in a generic method, and act as placeholders for the types of the arguments passed to the generic method (actual type arguments).
- A generic method's body is declared like that of any other method.
- **Type parameters can represent *only reference types*—not primitive types.**

Generic Methods (cont.)

- When the compiler translates generic method printArray into Java bytecodes, it removes the type-parameter section and *replaces the type parameters with actual types*.
- This process is known as erasure.
- By default all generic types are replaced with type Object.
- So the compiled version of method printArray appears as shown in Fig. 20.4—there is only *one* copy of this code, which is used for all printArray calls in the example.

```java
public static void printArray(Object[] inputArray)
{
   // display array elements
   for (Object element : inputArray)
      System.out.printf("%s ", element);

   System.out.println();
}
```

Fig. 20.4 | Generic method `printArray` after the compiler performs erasure.

Generic Methods (cont.)

- *A generic method may be overloaded like any other method.*
- A class can provide two or more generic methods that specify the same method name but different method parameters.
- A generic method can also be overloaded by nongeneric methods.
- When the compiler encounters a method call, it searches for the method declaration that best matches the method name and the argument types specified in the call
  - an error occurs if two or more overloaded methods both could be considered best matches.

Generic Classes

- The concept of a data structure, such as a stack, can be understood ***independently*** of the element type it manipulates.
- Generic classes provide a means for describing the concept of a stack (or any other class) in a ty*pe-independent* manner.
- These classes are known as parameterized classes or parameterized types because they accept one or more type parameters.
  - A type parameter will be replaced by a real type when an object of the generic class is created.

Generic Classes (cont.)

```
public class Box<T> {
    // T stands for "Type"
    private T ob;

    public void set(T o) { ob = o; }
    public T get() { return ob; }
}
```

- To reference the generic Box class from within your code, you must perform a generic type invocation, which replaces T with some concrete value, such as Integer:

```
Box <Integer> integerBox;
```

- To instantiate this class, use the new keyword, as usual, but place <Integer> between the class name and the parenthesis:

```
Box<Integer> integerBox = new Box<Integer>();
```

Introduction to Collections and Class ArrayList

- Java API provides several predefined data structures, called collections, used to store groups of related objects.
  - Each provides efficient methods that organize, store and retrieve your data without requiring knowledge of how the data is being stored.
  - Reduce application-development time.
- Arrays do not automatically change their size at execution time to accommodate additional elements.
- `ArrayList<T>` (a generic classes from package `java.util`) can dynamically change its size to accommodate more elements.
  - `T` is a placeholder for the type of element stored in the collection.
  - This is similar to specifying the type when declaring an array, except that only nonprimitive types can be used with these collection classes.

Collections and Class ArrayList (cont.)

Method	Description
`add`	Adds an element to the end of the `ArrayList`.
`clear`	Removes all the elements from the `ArrayList`.
`contains`	Returns `true` if the `ArrayList` contains the specified element; otherwise, returns `false`.
`get`	Returns the element at the specified index.
`indexOf`	Returns the index of the first occurrence of the specified element in the `ArrayList`.
`remove`	Removes the first occurrence of the specified value.
`remove`	Removes the element at the specified index.
`size`	Returns the number of elements stored in the `ArrayList`.
`trimToSize`	Trims the capacity of the `ArrayList` to current number of elements.

Fig. 7.23 | Some methods and properties of class `ArrayList<T>`.

Collections and Class ArrayList (cont.)

- An `ArrayList`'s **capacity** indicates how many items it can hold without growing.
- When the `ArrayList` grows, it must create a larger internal array and copy each element to the new array.
  - This is a time-consuming operation. It would be inefficient for the `ArrayList` to grow each time an element is added.
  - An `ArrayList` grows only when an element is added and the number of elements is equal to the capacity—i.e., there is no space for the new element.

- Method `add` adds elements to the `ArrayList`.
  - One-argument version appends its argument to the end of the `ArrayList`.
  - Two-argument version inserts a new element at the specified position.
  - Collection indices start at zero.

Collections and Class ArrayList (cont.)

```
import java.util.ArrayList;
public class ArrayListCollection
{
   public static void main(String[] args)
   {
      // create a new ArrayList of Strings with an initial capacity of 10
      ArrayList<String> items = new ArrayList<String>();
      items.add("red"); // append an item to the list
      items.add(0, "yellow"); // insert "yellow" at index 0
      // header
      System.out.print(
         "Display list contents with counter-controlled loop:");
      // display the colors in the list
      for (int i = 0; i < items.size(); i++)
         System.out.printf(" %s", items.get(i));
      // display colors using enhanced for in the display method
      display(items,
         "%nDisplay list contents with enhanced for statement:");
      items.add("green"); // add "green" to the end of the list
      items.add("yellow"); // add "yellow" to the end of the list
      display(items, "List with two new elements:");
      items.remove("yellow"); // remove the first "yellow"
      display(items, "Remove first instance of yellow:");
      items.remove(1); // remove item at index 1
      display(items, "Remove second list element (green):");
```

Collections and Class ArrayList (cont.)

```
      // check if a value is in the List
      System.out.printf("\"red\" is %sin the list%n",
         items.contains("red") ? "": "not ");

      // display number of elements in the List
      System.out.printf("Size: %s%n", items.size());
   }

   // display the ArrayList's elements on the console
   public static void display(ArrayList<String> items, String header)
   {
      System.out.print(header); // display header

      // display each element in items
      for (String item : items)
         System.out.printf(" %s", item);

      System.out.println();
   }
} // end class ArrayListCollection
```


Collections and Class ArrayList (cont.)

```
Display list contents with counter-controlled loop: yellow red
Display list contents with enhanced for statement: yellow red
List with two new elements: yellow red green yellow
Remove first instance of yellow: red green yellow
Remove second list element (green): red yellow
"red" is in the list
Size: 2
```

Raw Types

- It's also possible to instantiate a generic class without specifying a type argument, as follows:
  - ```
    // no type-argument specified
    ArrayList  objectList = new ArrayList();
    ```
  - objectList has a raw type
  - The compiler implicitly uses type **Object** throughout the generic class for each type argument.
  - The preceding statement creates an ArrayList that can store objects of *any* type.
  - Important for *backward compatibility* with prior Java versions.
  - Raw-type operations are **unsafe** and could lead to **exceptions**.
    ```